Comparisons
NoSql Databases
vs
Standard OLTP Databases

Esilv 2025

Arnaud.nauwynck@gmail.com

Overview

- Standard in OLTP :
  - ACID
  - Read isolation (read consistent, mvcc)
  - Sql SELECT + INSERT, UPDATE, DELETE
  - Transaction WAL Files
  - Modifying Binary files
  - Using B-Trees, and Indexes
  - Fast lookup by rowid/addresses in file blocks

- NOT Anymore in NoSql Databases !!
  - Commit, using « _temporary/* » directories?
  - Read isolation ?
  - Overwrite Files… delete + re-create new
  - No Indexes
  - …

« Si tu as besoin de quelque chose,
appelle moi, je te dirais comment t’en passer »

« If you need anything, call me,
I'll tell you how to do without it. »



ACID

A = Atomic

C = Consistent

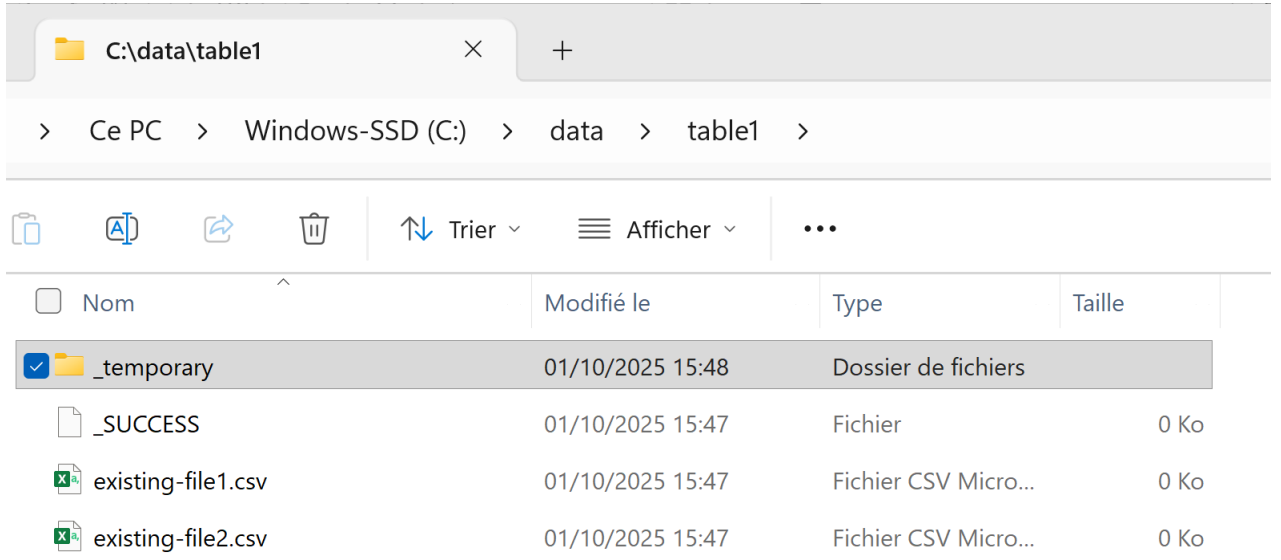
I = Isolated

D = Durable

ACID, A = Atomic

Atomic = All (commit) or Nothing (rollback)

Using « _temporary » dirs



Hidden files & Dirs: starting with « _ » or « . »

Example of hidden files:

« _SUCCESS » empty marker file for Hadoop for success end of task
« .file1.csv.crc » «SHA-256 » checksum file, for detecting corrupted data files

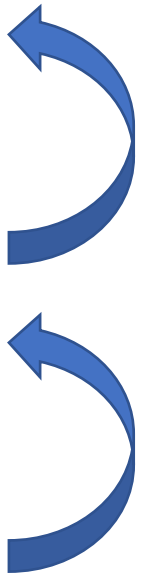
Example of hidden directories

« _temporary / » during task, then removed at end

Slow Write to Hidden temporary directory… OK then « atomic ?» delete all OLD files + rename all NEW files

/ table1 /
+ existing-file1.csv
+ existing-file2.csv
…
+ existing-file50.csv

/_temporary /
+ new-file1.csv
+ new-file2.csv
…
+ new-file200.csv



```
# step 1…50 : Remove ALL OLD …
rm existing-file1.csv;
rm existing-file2.csv;
..
rm existing-file50.csv;
```

```
# step 51…250 : Rename ALL NEW  …
mv _temporary/new-file1.csv   ./new-file1.csv;
mv _temporary/new-file2.csv   ./new-file2.csv;
..
mv _temporary/new-file200.csv  ./ new-file200.csv;
```

Is It really Atomic ?

Example at SG, we have some huge spark-jobs
The « Atomic » renaming of files takes too long => Timeout / Failure inside the « commit » !?

There might be still some « _temporary » remaining dir, partially some new files added …. And all old files deleted

The commit is SO LONG … you may read partially « uncommitted » files, but not all

ACID, C = Consistent

```
val dataset = spark.read.load(« /table1 »)

dataset.count();   // => OK, first read success

…
…   then processing for 20 minutes …
…
```



```
// another process running
val newDataset = …
newDataset.write.save(« /table1 »);
```

```
// … then actual read again the expected same files !
dataset.count();
// ➔ throw new FileNotFoundException(« …File ‘file1’ does not exist »);
```



Spark does not even return the « new refreshed » result …it just fails

ACID, I = Isolated

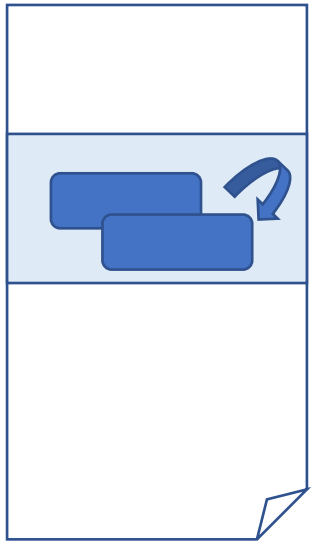
it is not Atomic, nor Consistent … it is also not Isolated

In particular, spark tries to be ACID, but per directory
… Suppose you are writting to several partitions (1 partition = 1 directory in Hive MetaStore)

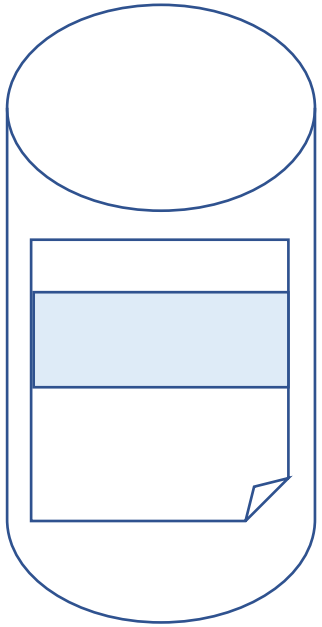
⇒ You may see some committed partitions (so partially committed task result), but not others
⇒ Even from different writer processes

Writing Row UPDATE in Standard OLTP Databases

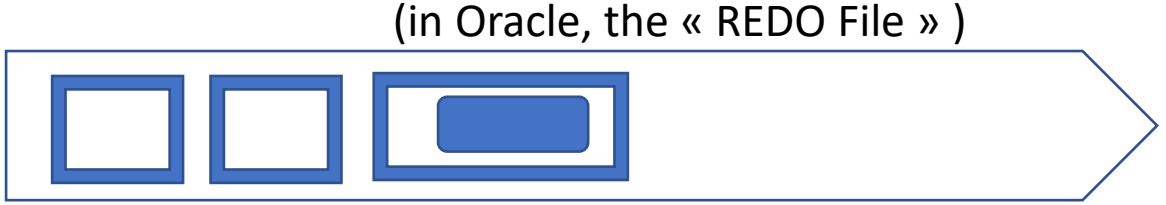
Modify directly
the RAM memory
ROW

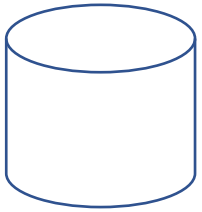


The Modified
Block is marked as « Dirty »
… to be saved later to Disk



Write append the Commit changes in the WAL Transactional Log File
(in Oracle, the « REDO File »)

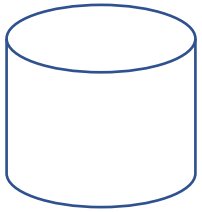




Write append +
Immediate Sync-FLUSH File

In Oracle… also write the OLD ROW in the « UNDO File »

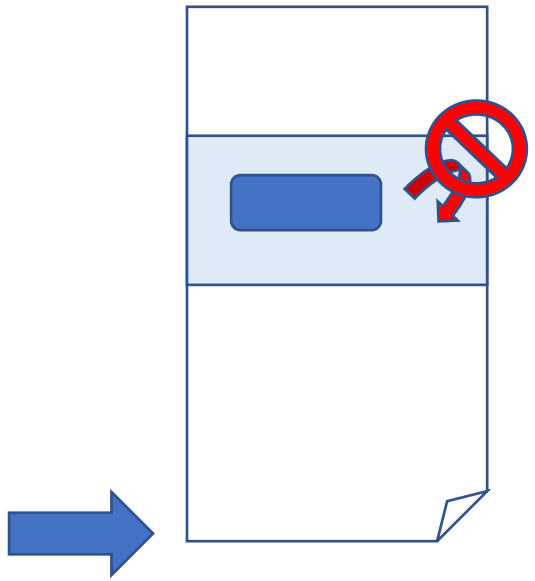




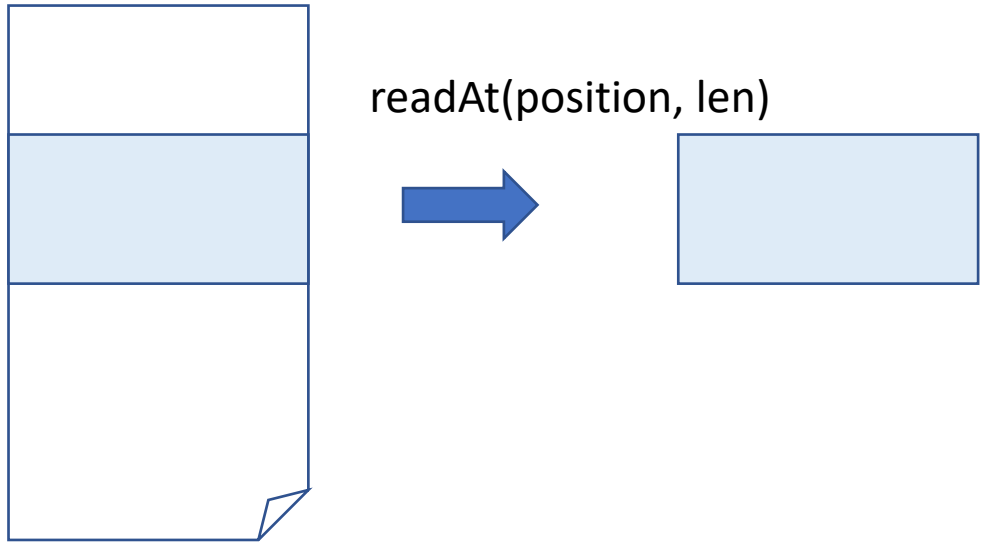
In BigData Storage : No « lseek() » + « writeAt() » File !!! (and DataSet are Immutable)

CAN NOT Modify directly
a File in the « middle »

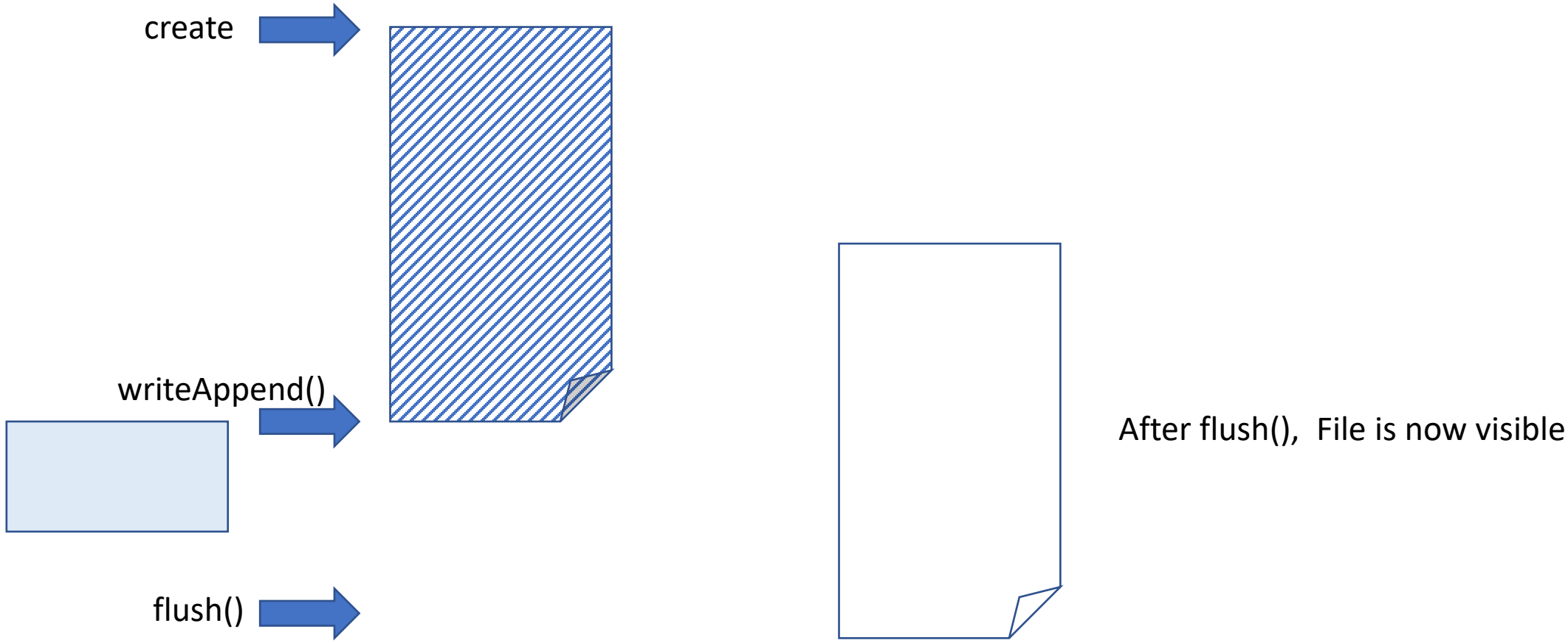
CAN Only « Write Append » to File,
or Delete+RE-Create it from 0!



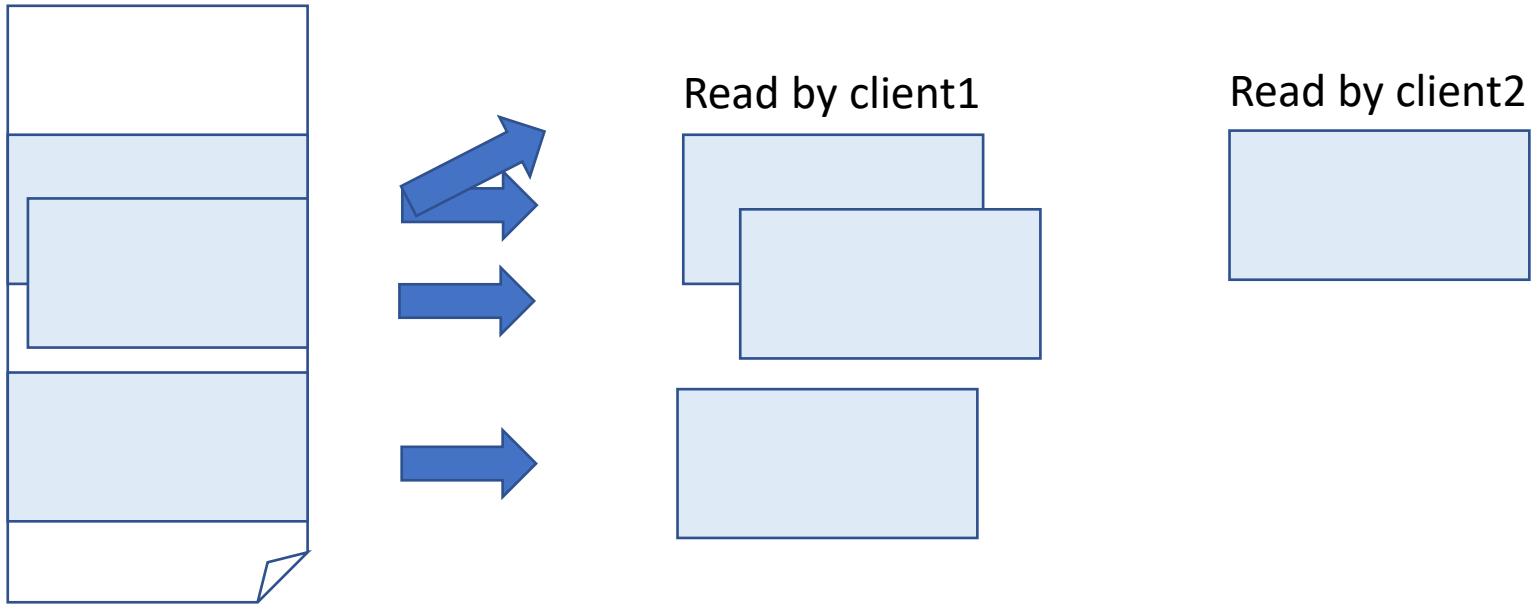
Notice… It is always possible to readAt() File



Writer : can only writeAppend into created temporary « opened-file » THEN Flush



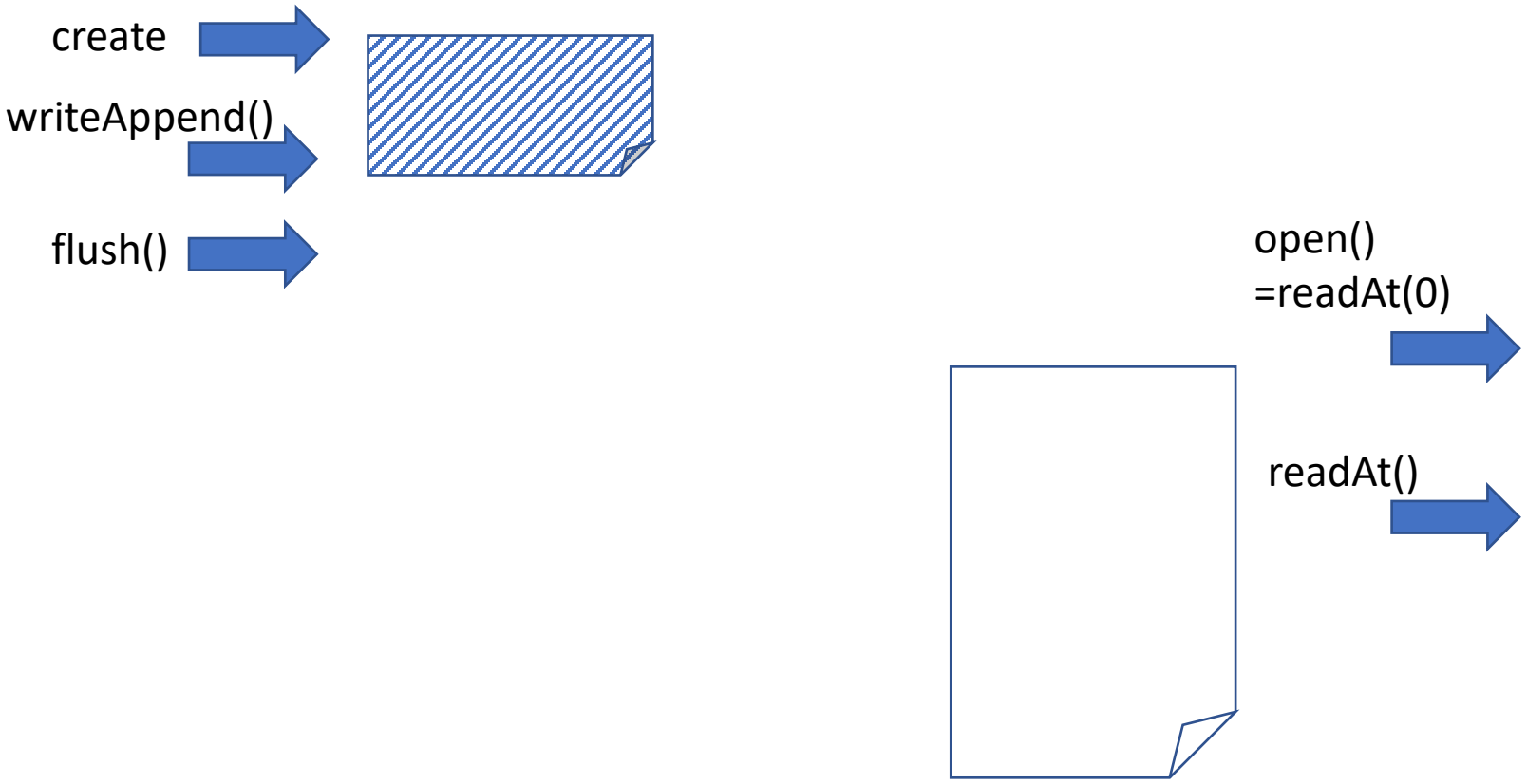
Reads can be Parallelized/Concurrent (several Threads)



In general, Reads will be aligned to 128Mo offsets

Several Clients can read the same region in « duplicate reads »

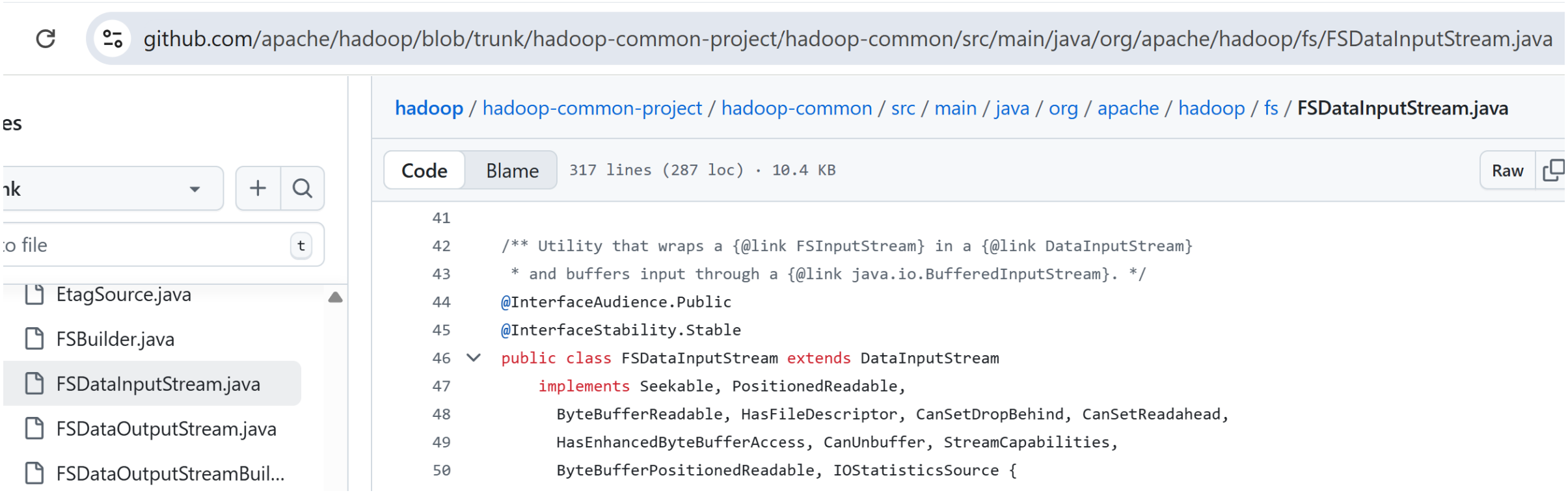
Reader can only see Fully « Written+Flushed » Files, not intermediate write still-in-progress



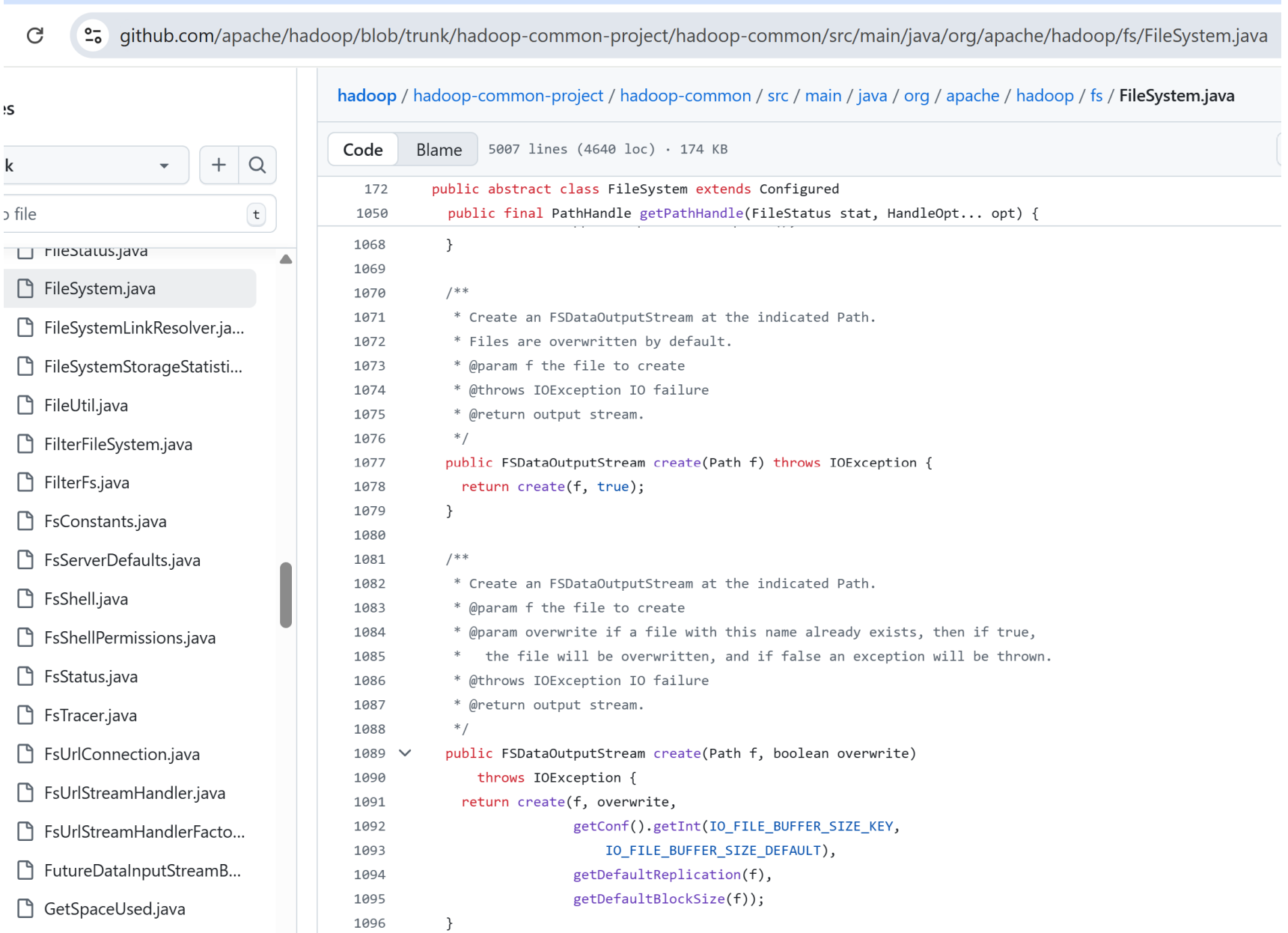
See Hadoop FileSystem Read API: FileSystem.open(Path) -> FSDataInputStream



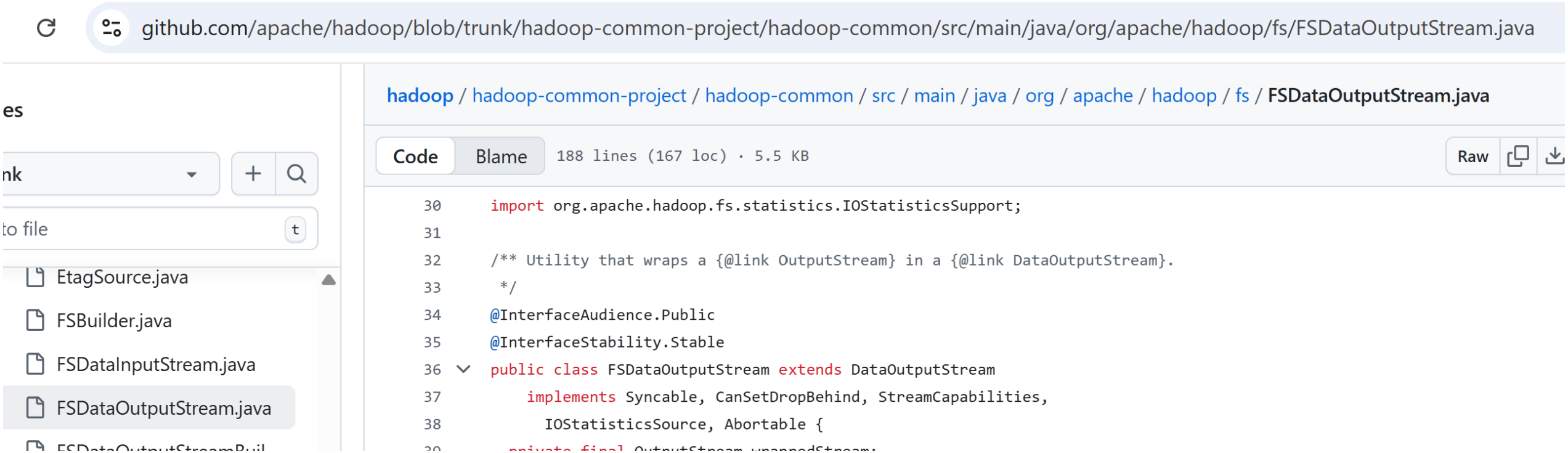
FSDataInputStream = InputStream + Seekable



See Hadoop FileSystem Write API: create()



See FSDataOutputStream = only write+flush (and optionnaly « dropBehind() »



NO write lseek()
=>
NO Hope of « backporting » standard Databases (Postgresql,Redis,…) to Hadoop !

Notice… « MMAP » File

Most databases use   Memory-Mapped-Files,

directly visible as virtual Memory from the Linux OS

faster than sequential reading files   (reading per block / pages)

Then Flush() dirty pages

….

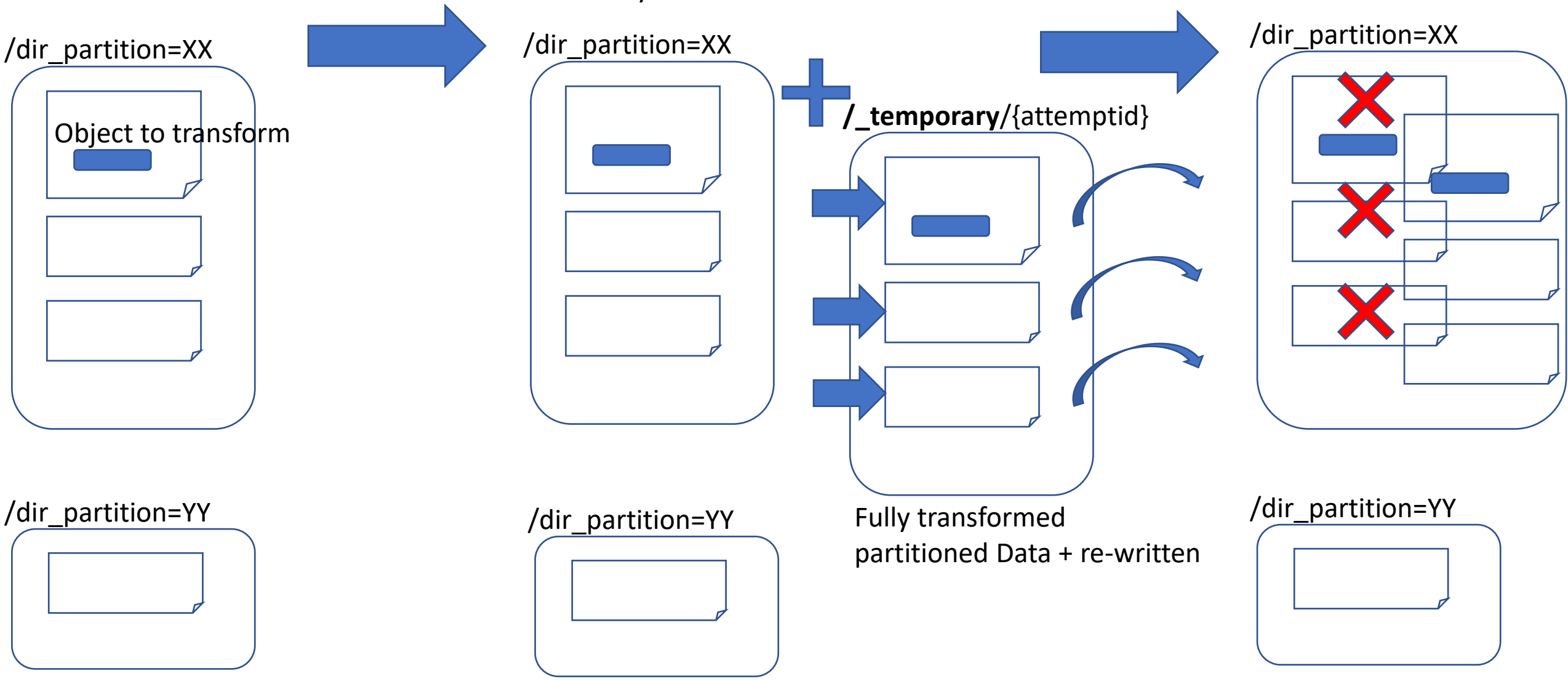
Not strictly necessary, but OS optim for disk access

Some database (like Oracle) prefer direct root access to « /dev/sda » block devices
(No FileSystem abstraction)

Updades Strategy : Overwrite per Partition(=Dir)

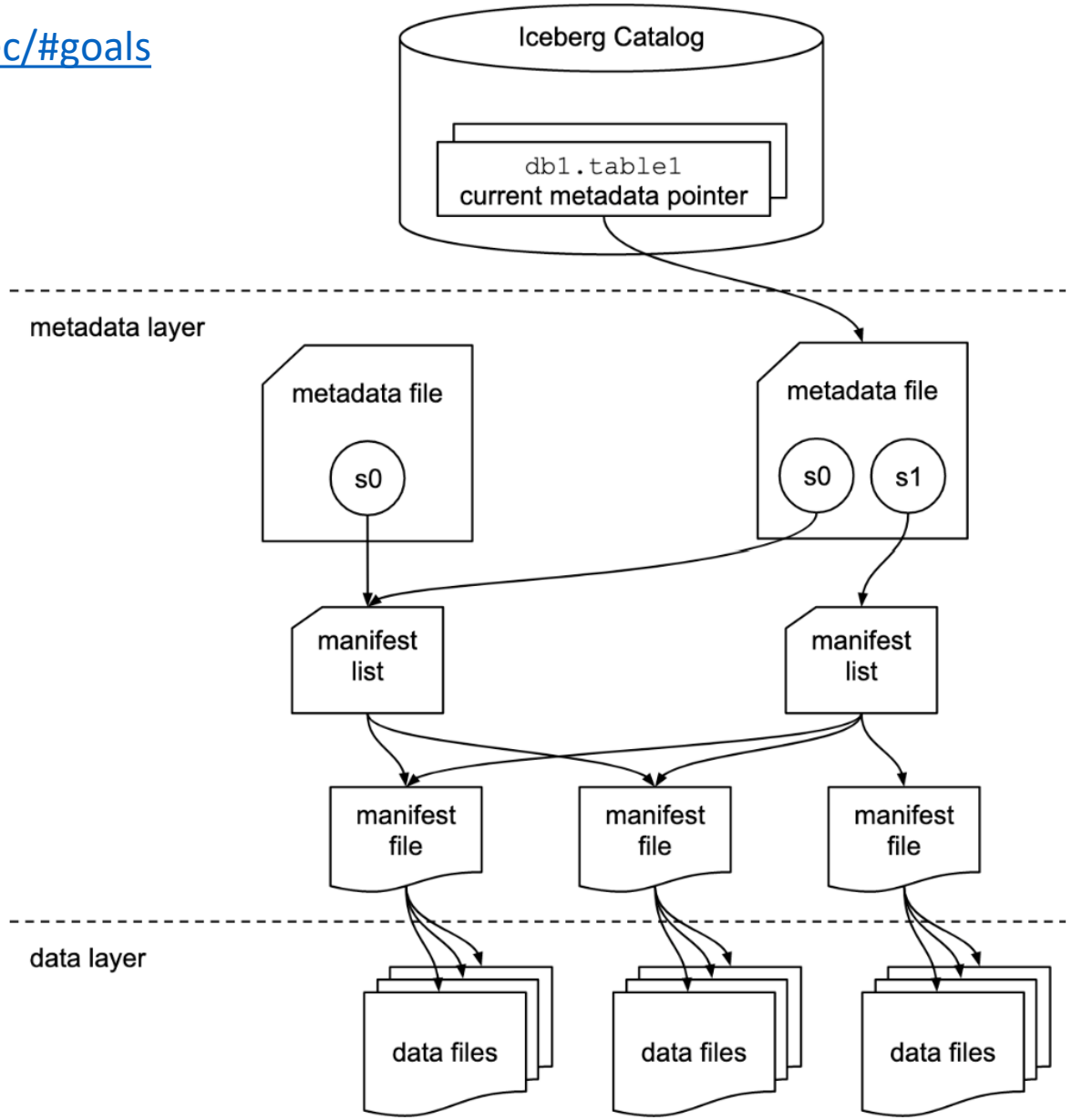
.map() transformation of data in 1 Partition
= FULL UPDATE COPY of a Directory

Overwrite: delete OLD files,
Rename _temporary/* to NEW files



Iceberg Strategy for Updates … like Git commit-blobs

https://iceberg.apache.org/spec/#goals



Iceberg => Allow Read Isolation / Consistency !!

Solve the problems of concurrent read/write problems occurring with Hive (Partitioned) Tables

Iceberg Bonus: Time Travel Queries, Branch, ..

Same as in Oracle « SELECT AS OF … »

Same as in Git

Iceberg Performance Problems

When Updating Row(s) =>

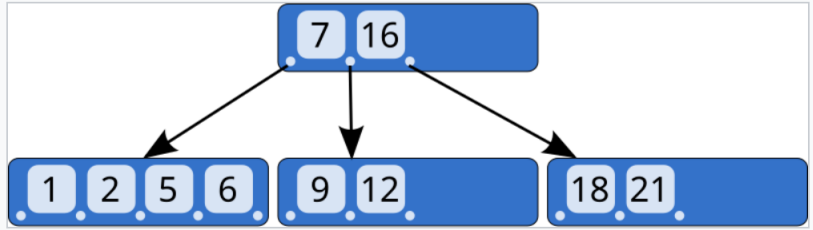
need to Fully Duplicate the File (Copy on Write)
OR
add a new File containing only the new Row + file marking the ids to « delete » (Merge on Read)

…. Explosions of Files number
OR
…. Explosion of compute treatments for read

B-Tree … Log(N) to lookup a Row by ID

Node structure [edit]

As with other trees, B-trees can be represented as a collection of three types of nodes: *root*, *internal* (a.k.a. interior), and *leaf*.



A B-tree (Bayer & McCreight 1972) of order 5 (Knuth 1998).

Note the following variable definitions:

- K : Maximum number of potential search keys for each node in a B-tree. (this value is constant over the entire tree).
- pt_i : The pointer to a child node that starts a sub-tree.
- pr_i : The pointer to a record that stores the data.
- k_i : The search key at the zero-based node index i.

In B-trees, the following properties are maintained for these nodes:

- If k_i exists in any node in a B-tree, then k_{i-1} exists in that node where $i \geq 1$.
- All leaf nodes have the same number of ancestors (i.e., they are all at the same depth).

Each internal node in a B-tree has the following format:

Internal node structure

pt_0	k_0	pt_1	pr_0	k_1	pt_2	pr_1	...	k_{K-1}	pt_K	pr_{K-1}

In BigData: No B-Tree files
…only PARQUET « Splittable + Columnar » files

In BigData … generally NOT a good idea to keep petabyte of file in RAM

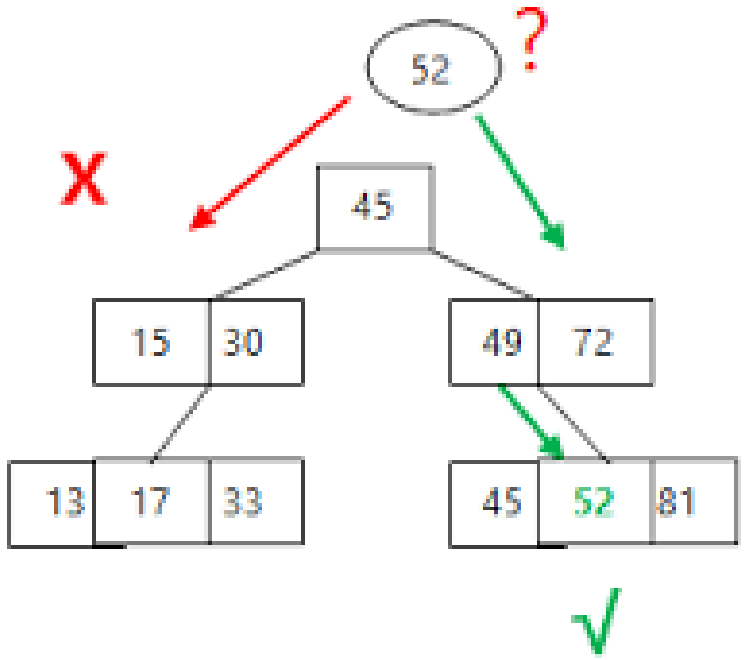
Not even Page Blocks of these files

… B-TREE lose their interrest if they are not « cached », and access per pages

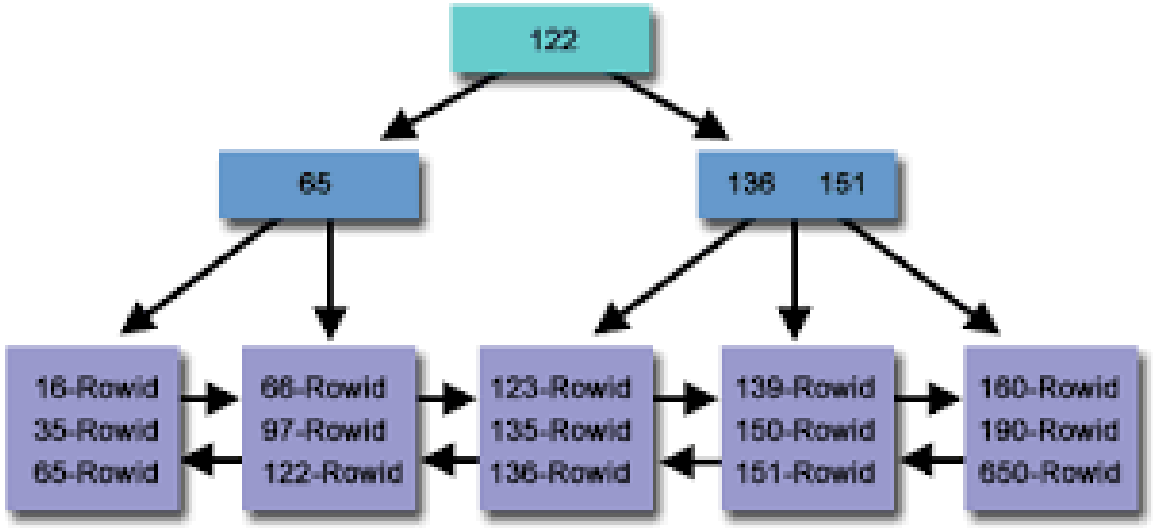
Better to read compressed files, rather than follow binary « pointers » within files

B-TREE => efficient
(IF-AND-ONLY-IF mostly cached in memory)
lookup Row by ID : log(N)

Figure 3: Search B-Tree - Order 3



B-TREE : Efficient to do « Index Range Scan »



In BigData … prefer Parallelizing Full Scan, or Parallelizing Optimized/Partial Scans

Cf Course on Parquet Optimizations:

1/ Partition Pruning
2/ Column Pruning
3/ Predicate Push Down
4/ Using Statistics ..
5/ Using Bloom Filters

TODO - WORK IN PROGRESS…